Relatório Técnico — PMW Tasks

Disciplina Programação e Modelagem para Web (PMW) — FSG · Prof. Adelcio Biazi · João Alfredo Tamagno

1. Estrutura do projeto

Projeto em HTML, CSS e JavaScript puro (sem frameworks nem build), com os dados persistidos em `localStorage`. Os arquivos JS são separados por responsabilidade, e o layout comum (header/nav/footer) é reaproveitado entre as páginas via Web Components, evitando duplicação de HTML:

```
projetoPMW/
├── index.html          Login
├── registro.html       Cadastro de usuário
├── home.html           Lista de tarefas do usuário logado
├── createTask.html     Criação de tarefa
├── perfil.html          Dados do usuário + troca de senha
├── css/style.css        Design system (cores, raio, sombra) e estilos
└── js/
    ├── storage.js       Único ponto de acesso ao localStorage
    ├── layout.js        Web Components do header/nav/footer
    ├── auth.js          Login, registro, logout, guarda de sessão
    ├── tasks.js         CRUD de tarefas
    └── profile.js       Perfil e troca de senha
```

2. Principais decisões técnicas

- **Modularização por responsabilidade:** em vez de um único script, cada arquivo cuida de uma parte (acesso a dados, autenticação, tarefas, perfil, layout), facilitando manutenção e leitura.
- **`storage.js` como única porta de entrada ao `localStorage`:** nenhum outro arquivo acessa `localStorage` diretamente, centralizando as chaves usadas (`usuarios`, `tarefas`, `userLogado`).
- **Web Components nativos (`customElements`)** para header/nav/footer: cada página só declara `<site-header>`/`<site-footer>`. Optou-se por essa abordagem (em vez de `fetch` de HTML parcial) porque o projeto pode ser aberto direto do disco (`file://`), onde `fetch` local é bloqueado por CORS.
- **Design system em variáveis CSS** (cor primária, raios de borda, sombras) aplicado de forma consistente a botões, cards, inputs e badges de status/urgência, com paleta pensada para transmitir foco/produtividade.
- **Responsividade orientada a conteúdo:** abaixo de 600px a tabela de tarefas vira uma lista de cards empilhados (em vez de rolagem lateral) e o footer é simplificado; o header usa `position: sticky` em qualquer tamanho de tela.

3. Demonstração de como cada requisito foi atendido

Requisito	Onde	Como
Manipulação de objetos HTML e eventos com DOM	`auth.js`, `tasks.js`, `profile.js`	Listeners de `submit` nos formulários de login/registro/tarefa/senha; clique nas linhas da tabela e no botão "Excluir"; modal de confirmação criado e removido dinamicamente.
Uso de vetores/matrizes	`storage.js`, `tasks.js`	Usuários e tarefas são arrays manipulados com `filter`, `find`/`findIndex`, `map` e `forEach` (ex.: filtrar tarefas do usuário logado, localizar a tarefa clicada).
Criação de objetos em tempo de execução	`tasks.js`	`document.createElement` gera as linhas da tabela e o modal de confirmação; os objetos de tarefa/usuário recebem propriedades (`id`, `done`, `user`) atribuídas dinamicamente ao serem criados.

Requisito	Onde	Como
CRUD completo com `localStorage`	`storage.js` + `auth.js`/`tasks.js`/`profile.js`	**Create:** cadastro de usuário e criação de tarefa. **Read:** login e listagem de tarefas. **Update:** concluir tarefa e trocar senha. **Delete:** excluir tarefa (com confirmação em modal).
Design responsivo e amigável com CSS	`style.css`	`@media (max-width: 600px)` transforma a tabela em cards e compacta o footer; header fixo (`sticky`); inputs/botões com estados de foco e hover.

Repositório: github.com/Audamon/PWMTasks